

Backend Engineering Master Knowledge Map

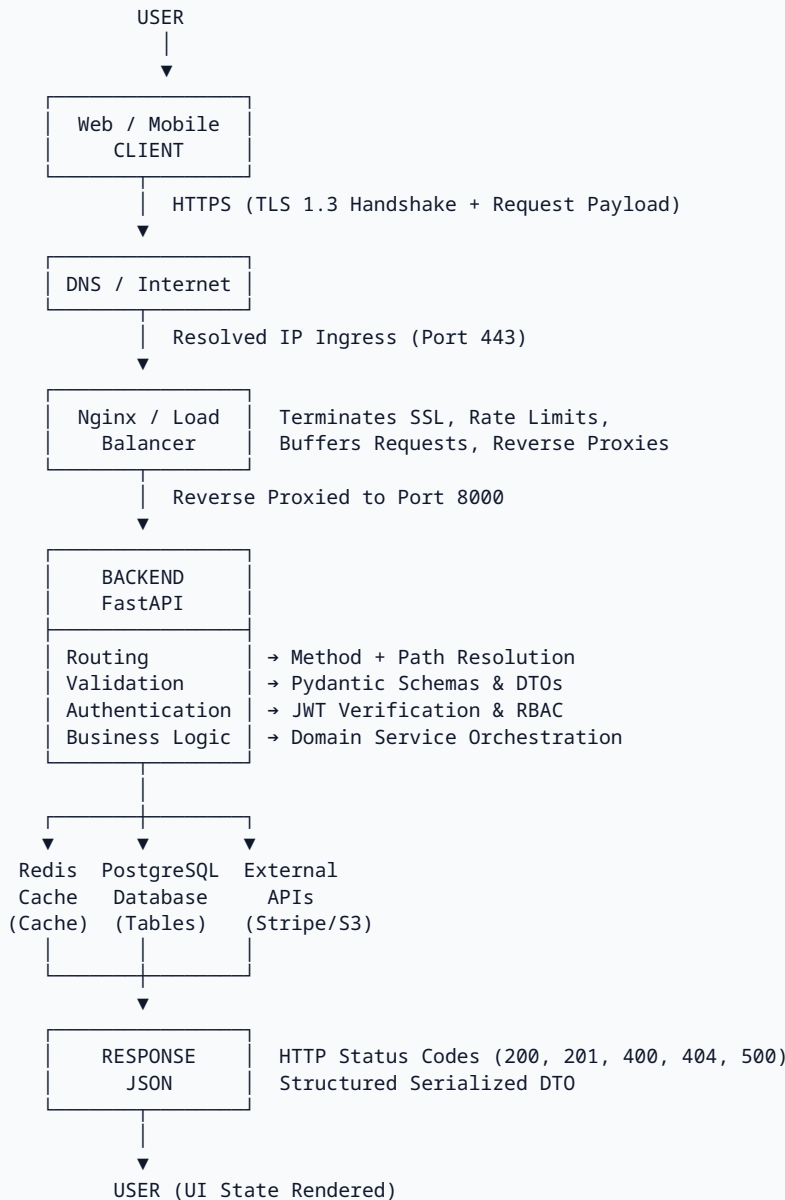
Interactive 18-Domain Production Blueprint & Real-World Request-Response Lifecycle

Structure: 18 Core Disciplines (Foundations to Distributed)
Format: Interactive Index & Cross-Linked Domains

Anchor: Request Pipeline: User → Nginx → FastAPI → DB

 Quick Navigation Index (Click any section to jump directly)

- | | | |
|--------------------------------|--------------------------------|-------------------------|
| → Visual Request Journey | 07. Data Access & ORM | 13. Linux & Server |
| 01. Computer & OS Fundamentals | 08. Authentication & Security | 14. Deployment & DevOps |
| 02. Networking | 09. Performance & Concurrency | 15. Cloud Computing |
| 03. Web Fundamentals | 10. Background & Communication | 16. Observability |
| 04. API Engineering | 11. File & External Services | 17. System Design |
| 05. Backend Application | 12. Testing & Quality | 18. Distributed Systems |
| 06. Database & Data | | |



Layer 1: Fundamentals, Networking & APIs

01 Computer & OS Fundamentals

[↑ Top](#)

- **CPU / RAM / Storage:** Hardware execution pipelines, volatile memory vs. non-volatile disk access latencies.
- **Processes / Threads:** Isolated memory address space vs. shared thread memory, context switching overhead.
- **Filesystem:** POSIX permissions, file descriptors (everything is a file in Unix), buffering.
- **Environment:** Environment variables (12-factor config), process isolation, standard streams (stdin/out/err).

03 Web Fundamentals

[↑ Top](#)

- **Client & Server:** Separation of UI state and backend business invariants over network boundaries.
- **Request / Response:** Request lines, headers, payload body, MIME types (application/json).
- **HTTP / HTTPS:** Stateless protocol, TLS cryptographic handshake (certificate validation, AES symmetric session).
- **Headers, Cookies & Sessions:** Stateful auth cookies (HttpOnly, Secure, SameSite), session storage.

02 Networking

[↑ Top](#)

- **Internet & Routing:** Packet-switched networking, Autonomous Systems (AS), gateways, NAT.
- **IP & Ports:** IPv4 (32-bit), IPv6 (128-bit), privileged ports (<1024), dynamic port ranges.
- **TCP / UDP:** TCP 3-way handshake, flow control, ordered delivery vs. UDP connectionless speed.
- **DNS:** Domain to IP resolution, A/AAAA records, CNAME, TTL caching, recursive lookup hierarchy.

04 API Engineering

[↑ Top](#)

- **REST & Resources:** Plural resource modeling (/users, /orders), standard CRUD verbs.
- **Parameters:** Path (identity), Query (filters, search, pagination), Body (payload mutations).
- **Status Codes:** 2xx (Success), 3xx (Redirect), 4xx (Client error), 5xx (Internal backend crash).
- **API Design & Idempotency:** Safe idempotent operations (GET, PUT, DELETE) vs. non-idempotent (POST), API versioning.

Layer 2: Application Architecture, Data & Security

05 Backend Application

[↑ Top](#)

- **Routing & Controllers:** Ingress HTTP routing to designated controllers/endpoints.
- **Validation:** Pydantic type safety, sanitizing untrusted client input before logic execution.
- **Business Logic & Services:** Domain rules decoupled from HTTP presentation and database schemas.
- **Dependency Injection & Errors:** Injecting database sessions, uniform structured exception handlers.

07 Data Access

[↑ Top](#)

- **ORM (SQLAlchemy):** Mapping entity objects to relational schemas with parameterized SQL safety.
- **Database Sessions:** Unit of Work pattern, coordinating commit and rollback boundaries.
- **Connection Pooling:** Mitigating TCP handshake overhead by sharing pre-warmed database sockets.
- **Alembic / Migrations:** Versioned, reversible database schema evolution tracked in Git.

06 Database & Data

[↑ Top](#)

- **SQL & Relational DBs:** DDL, DML, relational integrity, foreign keys, cascade rules.
- **PostgreSQL:** Advanced relational engine (JSONB, full-text search, MVCC isolation).
- **Indexes:** B-Trees, GiST, GIN, composite indexes, query execution analysis (EXPLAIN ANALYZE).
- **Transactions & ACID:** Atomicity, Consistency, Isolation levels, Durability via Write-Ahead Logs (WAL).

08 Authentication & Security

[↑ Top](#)

- **AuthN vs. AuthZ:** Identity verification vs. Role-Based Access Control (RBAC) permissions.
- **Password Hashing:** Slow cryptographic algorithms (Argon2id, bcrypt) with unique salts.
- **JWT & OAuth:** Stateless asymmetric tokens (access/refresh lifecycle), third-party delegated authorization.
- **Web Security:** CORS configuration, CSRF prevention, SQL injection protection, rate limiting.

Layer 3: Performance, Task Queues & Quality

09 Performance & Concurrency

[↑ Top](#)

- **Async / Await:** Non-blocking cooperative multitasking powered by the asyncio event loop.
- **Threads vs. Processes:** Thread pool for I/O waits vs. multiprocessing to bypass Python's GIL for CPU work.
- **Redis Caching:** Cache-aside strategy, TTL expiration, invalidation policies (LRU/LFU).
- **Optimization:** Hot-path code profiling, reducing memory allocations, mitigating N+1 database queries.

11 File & External Services

[↑ Top](#)

- **File Uploads & Object Storage:** Presigned URLs directly to Amazon S3 to bypass application RAM buffers.
- **Third-Party APIs:** Integrating payment gateways (Stripe), email providers (SendGrid), and SMS services (Twilio).
- **Resilience Patterns:** Exponential backoff retries, request timeouts, circuit breakers preventing cascade crashes.

10 Background & Communication

↑ Top

- **Background Jobs:** Offloading heavy operations (emails, PDF generation, bulk exports) from HTTP thread.
- **Task Queues:** Celery / ARQ task producers, persistent worker pools, dead-letter queues (DLQ).
- **Message Brokers:** RabbitMQ (AMQP message routing) and Apache Kafka (distributed event log streams).
- **Webhooks:** Asynchronous outbound HTTP callbacks with retry and signature verification mechanisms.

12 Testing & Quality

↑ Top

- **Testing Pyramid:** Unit tests (pure functions), Integration tests (DB repository), and E2E API tests.
- **pytest:** Fixtures for test setup/teardown, mock objects for external APIs, transaction rollbacks for clean tests.
- **Coverage & Contracts:** Test coverage metrics, API schema validation, regression testing in CI.

Layer 4: Infrastructure, Cloud & Distributed Architecture

13 Linux & Server Management

↑ Top

- **Linux Substrate:** SSH key authentication, shell scripting, POSIX filesystem standards.
- **System Administration:** systemd service orchestration, signal handling (SIGTERM, SIGKILL).
- **Server Diagnostics:** Process monitoring (top, htop), networking inspection (netstat, curl).

14 Deployment & DevOps

↑ Top

- **Docker:** Containerization, multi-stage Dockerfiles, Docker Compose for local microservices.
- **Nginx:** Edge reverse proxy, SSL termination, static file delivery, round-robin load distribution.
- **CI/CD:** GitHub Actions automated linting, testing, image building, and automated zero-downtime deploys.

15 Cloud Computing (AWS)

↑ Top

- **Compute & Networking:** EC2, ECS container runners, VPC, private subnets, security groups, NAT gateways.
- **Managed Data:** AWS RDS PostgreSQL with automated snapshots, ElastiCache Redis, S3 object stores.
- **IAM & DNS:** Route 53 domain management, least-privilege IAM policies, ALB (Application Load Balancer).

16 Observability

↑ Top

- **Structured Logging:** Machine-readable JSON logs tagged with request correlation IDs (Trace IDs).
- **Metrics & Monitoring:** Prometheus collection, Grafana dashboards tracking RED metrics (Rate, Errors, Duration).
- **Tracing & Alerting:** OpenTelemetry distributed spans across microservices, PagerDuty / Slack alerts on 5xx surges.

17 System Design

↑ Top

- **Scalability & Availability:** Horizontal scaling of stateless backends, eliminating Single Points of Failure (SPOF).
- **Database Scaling:** Read replicas, connection pooling (PgBouncer), vertical partitioning, sharding.
- **Reliability:** Graceful degradation, healthcheck endpoints, disaster recovery replication.

18 Distributed Systems

↑ Top

- **Microservices:** Autonomous services communicating over HTTP/gRPC or asynchronous message queues.
- **CAP Theorem:** The fundamental trade-off between Consistency, Availability, and Partition Tolerance.
- **Distributed Transactions:** Eventual consistency, Saga patterns, distributed locking via Redis/etcd.

How to Use This Blueprint:

This map is an architectural compass, not a test you must pass before writing code. Keep it bookmarked: whenever you write an endpoint in FastAPI, configure a Dockerfile, or design a PostgreSQL schema, use this document to understand exactly which boundary of the system you are securing, optimizing, or deploying.